ENUNCIAT PROJECTE
Programació 3 – UAB

curs 2026-27

1. INTRODUCCIÓ

OpenStreetMap (OSM) és una organització sense ànim de lucre que distribueix dades d'informació geogràfica (mapes) de tot el món obtingudes de manera col·laborativa. Aquesta informació és oberta i es distribueix en format XML.





[1]

Per tal de conèixer el funcionament d'aquest servei, podeu utilitzar l'enllaç següent per visualitzar una regió concreta del mapa:

https://www.openstreetmap.org/#map=17/41.499477/2.109303

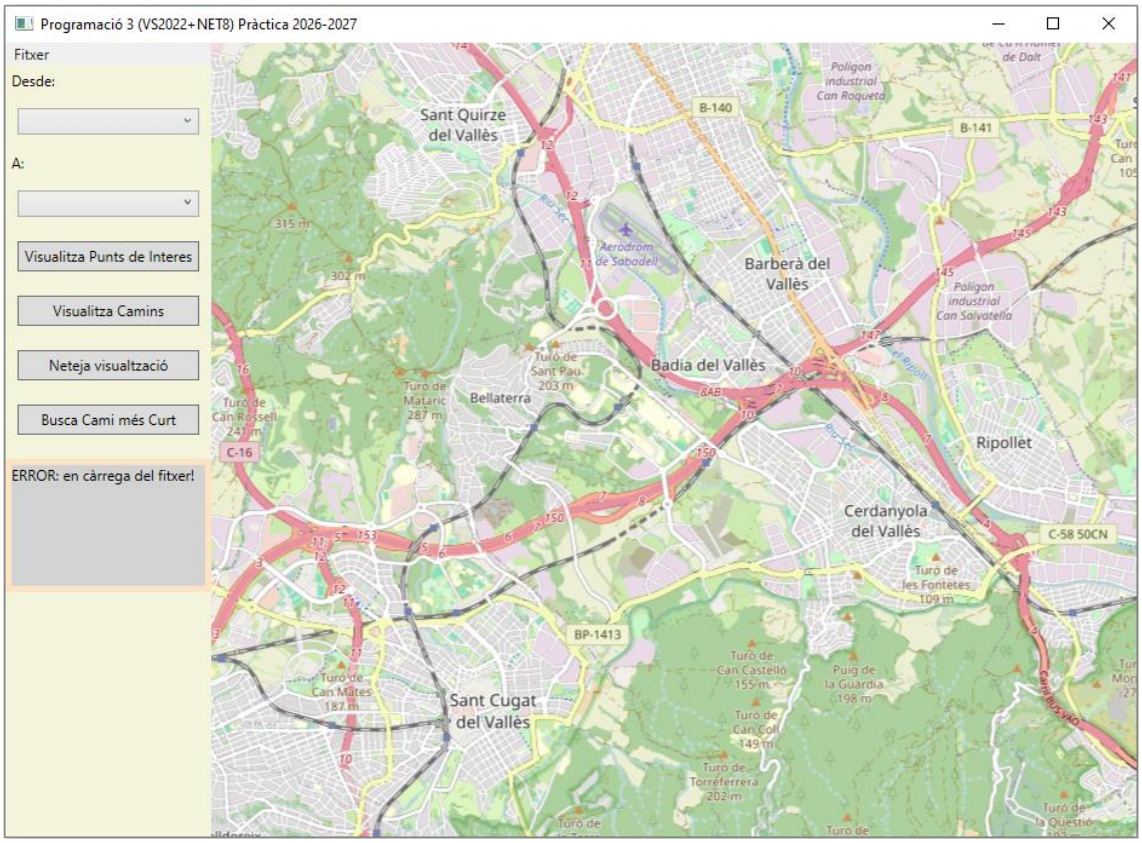
Aquest curs, la pràctica de Programació 3 es basa en la representació dels punts d'interès d'un mapa proporcionat per OSM. Així, a partir d'un fitxer XML que inclou informació de **punts d'interès** (POI) i de **carreteres** (vies) d'una secció del mapa, es duran a terme accions per processar la informació geogràfica i extreure'n dades. Les vies es representen com una llista de nodes connectats amb coordenades de latitud i longitud. En primer lloc, es llegirà un fitxer OSM per emmagatzemar-ne la informació. Seguidament, es representaran els punts d'interès i les carreteres sobre el mapa, fins a calcular i mostrar el camí més curt entre dos punts d'interès seleccionats mitjançant la interfície gràfica. L'aspecte clau del treball consistirà a dissenyar i implementar les estructures de dades més adients per emmagatzemar i accedir a la informació requerida de manera òptima.

[1] Imatges amb llicència CC, descarregades de Wikipedia: https://ca.wikipedia.org/wiki/OpenStreetMap

2. PROJECTE PART 1

La primera part de la pràctica consisteix a carregar un **fitxer XML** proporcionat per OpenStreetMap (fitxer OSM), llegir-lo i desar-ne la informació a les vostres pròpies estructures de dades. Aquesta informació es limitarà als nodes que formen les carreteres i als punts d'interès, descartant la resta de dades presents al fitxer.

El resultat a la interfície gràfica proporcionada és el que es mostra en la imatge següent:



Per assolir els objectius de la pràctica cal seguir rigorosament totes les instruccions proporcionades. Per a cada part del projecte es lliurarà un document d'enunciat i una guia de tasques (presentació). Al llarg de les seccions d'aquest document trobareu la informació necessària per al desenvolupament del projecte. En aquesta primera part, les tasques es divideixen en dos blocs: d'una banda, les Tasques 0 a 2 s'orienten a la configuració de l'entorn i a la comprensió de l'estructura del projecte, per la qual cosa es desenvoluparan en un codi independent i disposaran d'una guia pròpia; d'altra banda, les Tasques 3 a 9 formen part del lliurament avaluable i disposen de la seva corresponent guia. Als documents dels enunciats, les tasques apareixen clarament identificades dins de requadres de color.

2.0 ELS FITXERS XML DE OPENSTREETMAPS

Per començar, cal comprendre l'estructura dels **fitxers XML** proporcionats per **OSM.**

Un fitxer XML d'OSM es compon d'una sèrie d'elements, entre els quals us heu de centrar en els definits com a ***<node>*** i ***<way>.***

- ✓ Un ***<node>*** representa un punt a l'espai i defineix la seva latitud i longitud, així com un identificador, els quals constitueixen les dades fonamentals del fitxer.
- ✓ Un ***<way>*** és un conjunt ordenat de nodes, tot i que en aquest projecte només es processarà un subconjunt d'aquests.

A continuació es detallen els elements que es poden trobar en un fitxer XML d'OSM:

- **Capçalera:** Aquesta capçalera no la necessitareu per la pràctica.

```
<?xml version="1.0" encoding="UTF-8"?>
<osm version="0.6" generator="CGImap 0.8.5 (3025176 spike-
 07.openstreetmap.org)" copyright="OpenStreetMap and contributors"
 attribution="http://www.openstreetmap.org/copyright"
 license="http://opendatacommons.org/licenses/odbl/1-0/">
 <bounds minlat="41.5004500" minlon="2.1003300" maxlat="41.5043900"
 maxlon="2.1089500"/>
```

- **Nodes:** Els nodes es poden classificar en dues categories principals, de les quals els nodes de via poden pertànyer a tres subtipus diferents. Es manté la jerarquia següent:
    - Nodes que són nodes de Via (camí)
        - Nodes que no tenen tag.
        - Nodes que tenen un tag amb un atribut *highway*, *public_transport*, *access o entrance*.
        - Nodes amb tags on cap té l'atribut *name*.
    - Nodes que són Punts d'Interès

Els nodes de tipus Punt d'Interès són aquells que no pertanyen a cap via i que contenen l'etiqueta amb la clau *name*. El seu valor es mostrarà al mapa com a identificador del punt.

A continuació es mostren diversos exemples de nodes de via (observeu els elements en negreta):

Node sense etiquetes (node de via):

```
<node id="1644458226" visible="true" version="1" changeset="10769439"
 timestamp="2012-02-23T17:43:24Z" user="EliziR" uid="605366" lat="41.4992770"
 lon="2.1048901"/>
```

Node amb etiquetes sense cap atribut name (node de via):

```
<node id="522572539" visible="true" version="3" changeset="10769439"
 timestamp="2012-02-23T17:43:34Z" user="EliziR" uid="605366" lat="41.4993094"
 lon="2.1048279">
  <tag k="barrier" v="lift_gate"/>
 </node>
```

Node amb l'etiqueta highway, public_transport, entrance o access (node de via):

```
<node id="354873898" visible="true" version="8" changeset="103869767"
 timestamp="2021-04-29T21:04:44Z" user="caminsperfer" uid="10187213"
 lat="41.9662078" lon="2.7402486">
  <tag k="highway" v="traffic_signals"/>
 </node>

<node id="1287613323" visible="true" version="2" changeset="56244875"
 timestamp="2018-02-10T16:59:23Z" user="jqngarcia" uid="5126253"
 lat="41.5369864" lon="2.4303634">
  <tag k="bus" v="yes"/>
  <tag k="name" v="I. Català de Salut"/>
  <tag k="public_transport" v="stop_position"/>
 </node>
<node id="8647042980" visible="true" version="2" changeset="103643101"
 timestamp="2021-04-26T14:15:46Z" user="Interswd" uid="3414856"
 lat="41.4928225" lon="2.1422926">
  <tag k="access" v="yes"/>
  <tag k="emergency" v="defibrillator"/>
  <tag k="indoor" v="no"/>
  <tag k="name" v="AED"/>
  <tag k="opening_hours" v="24/7"/>
 </node>

<node id="2386533013" visible="true" version="3" changeset="64697386"
 timestamp="2018-11-20T13:22:48Z" user="Carto'Cité" uid="5426135"
 lat="48.8571334" lon="2.2908746">
  <tag k="entrance" v="main"/>
  <tag k="level" v="0"/>
  <tag k="name" v="Quai de Grenelle"/>
  <tag k="railway" v="train_station_entrance"/>
  <tag k="source" v="SNCF Transilien - Plans d'architecte et plan de
 communication - Prestation de saisie réalisée par Arx IT en hiver 2016"/>
 </node>
```

Finalment, un exemple de node de tipus Punt d'Interès:

Nodes amb l'etiqueta *<tag> name*, i sense les etiquetes *highway*, *public_transport*, *access* ni *entrance*. Aquest node correspon a un punt d'interès identificat visualment com a Facultat de Filosofia i Lletres:

```
<node id="1655719923" visible="true" version="7" changeset="57828127"
 timestamp="2018-04-05T09:22:58Z" user="Jesús Gómez" uid="6389"
 lat="41.5028375" lon="2.1075584">
  <tag k="name" v="Facultat de Filosofia i Lletres"/>
  <tag k="office" v="educational_institution"/>
  <tag k="website" v="https://www.uab.cat/lletres/"/>
 </node>
```

- **Way:** Indica una via que discorre per nodes que no són Punts d'Interès. Els elements fills del tag ***<way>*** determinen els nodes que en formen el recorregut. Cada node inclou informació de latitud i longitud, i es referencia mitjançant l'atribut ref.

NOTA: Per a la realització d'aquesta pràctica, es descartaran totes les vies que no incloguin l'etiqueta ***highway***.

A continuació es mostra un exemple d'element ***<way>*** utilitzat a la pràctica, on els nodes integrants s'identifiquen mitjançant l'atribut **ref**. Observeu que els dos primers identificadors de node (**"522572539", "1644458226"**) corresponen als dos nodes presentats en els primers exemples de la secció de nodes.

```
<way id="151708041" visible="true" version="10" changeset="54933536"
 timestamp="2017-12-26T16:13:54Z" user="FvGordon" uid="161619">
  <nd ref="522572539"/>
  <nd ref="1644458226"/>
  # [...]

  <tag k="foot" v="yes"/>
  <tag k="highway" v="service"/>
  <tag k="service" v="parking_aisle"/>
  <tag k="wheelchair" v="limited"/>
 </way>
```

El conjunt complet de característiques definides per l'XML d'OSM es pot consultar a la documentació oficial: Map features - OpenStreetMap Wiki.

TASCA 0: Anar a **https://www.openstreetmap.org/export**, exportar una regió d'una zona de manera manual, i observar-ne el contingut de ***<node>*** i ***<way>*** que el formen.

2.1 L'ESTRUCTURA XMLELEMENT

La interfície proporcionada llegeix un fitxer XML mitjançant una biblioteca dinàmica externa en C++ (*xerces-c_3_2D.dll*) i emmagatzema tota la informació en un vector de la STL. Aquest vector conté elements de tipus `XmlElement` (definit a `Common.h`). Cada `XmlElement` conté la informació dels nodes i les vies del fitxer i es defineix de la manera següent:

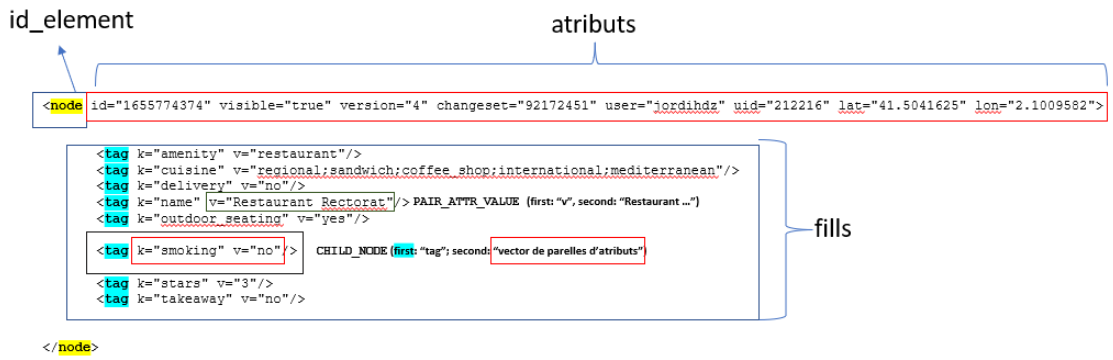
```
typedef std::pair<std::string, std::string> PAIR_ATTR_VALUE;
typedef std::pair<std::string, std::vector<PAIR_ATTR_VALUE>> CHILD_NODE;

typedef struct {
    std::string id_element;
    std::vector<PAIR_ATTR_VALUE> atributs;
    std::vector<CHILD_NODE> fills;
} XmlElement;
```

Aquesta estructura conté els següents tres camps:

- **`id_element`**: defineix el tipus d'element
- **`atributs`**: és un vector de parelles d'atributs d'aquest element
- **`fills`**: És el seu conjunt de **sub elements**.
    - Cada sub-element, consisteix en una parella a on el seu primer valor identifica el tipus d'aquest element, i el segon valor és un vector d'atributs, d'aquest sub-element.

Seguidament, podeu veure en un diagrama una comparació del que us trobareu en un fitxer d'OSM, identificant cadascuna de les variables:



Per donar-vos un exemple de com seria un Punt d'Interès, en aquest cas anomenat "El Roble", i que es tracta d'un restaurant que ofereix cuina regional i facilita l'accés a persones amb mobilitat reduïda, podeu veure com l'element `XmlElement` es construiria de la següent manera.

Primer tenim la seva representació en XML:

```
<node id="357101444" visible="true" version="3" changeset="77532251"
 timestamp="2019-11-25T15:26:03Z" user="MultiDavid2001b" uid="4946691"
 lat="41.4902204" lon="2.1406477">
  <tag k="amenity" v="restaurant"/>
  <tag k="cuisine" v="regional"/>
  <tag k="name" v="El Viejo Roble"/>
  <tag k="wheelchair" v="yes"/>
 </node>
```

I aquest és el codi que generaria les dades corresponents:

```
using namespace std;
XmlElement viejoRoble = {
	//id_element =
	"node",

	// atributs =
	{
		make_pair("id", "357101444"),
		make_pair("visible", "true"),
		make_pair("version", "3"),
		make_pair("changeset", "77532251"),
		make_pair("timestamp", "2019-11-25T15:26:03Z"),
		make_pair("user", "MultiDavid2001b"),
		make_pair("uid", "4946691"),
		make_pair("lat", "41.4902204"),
		make_pair("lon", "2.1406477")
	},

	// fills =
	{
		make_pair<string,vector<pair<string, string>>>
			("tag", {
				make_pair<string, string>("k", "amenity"),
				make_pair<string, string>("v", "restaurant")
		}),
		make_pair<string, vector<pair<string, string>>>
				("tag", {
					make_pair<string, string>("k","cuisine"),
					make_pair<string, string>("v", "regional")
```

```
                }),
                make_pair <string, vector<pair<string, string>>>
                        ("tag", {
                                make_pair<string, string>("k", "name"),
                                make_pair<string, string>("v", "El Roble")
                        }),
                make_pair <string, vector<pair<string, string>>>
                        ("tag", {
                                make_pair("k","wheelchair"),
                                make_pair("v", "yes")
                        })
        }
};
```

En cap cas heu de crear estructures de tipus XmlElement per a la realització de la pràctica, sinó únicament llegir-les. Tanmateix, és fonamental conèixer-ne la definició i l'estructura interna per poder processar adequadament la informació continguda.

Per comprovar que un element és un node, cal verificar el camp `id_element`. En aquest cas, cal emmagatzemar en una variable el valor de l'atribut `timestamp` i registrar en una variable booleana si el node és accessible per a persones amb mobilitat reduïda.

Per obtenir el valor de la clau d'una `etiqueta (tag)`, s'utilitzarà la funció `kvDeTag()` de la classe `Util`, descrita a la secció 2.6.

A continuació es mostra el codi d'exemple basat en la construcció anterior:

```
string tempsValor = "";
bool permetMobilitatReduida = false;
// Comprovem per id_element, que es tracta d'un node
if (viejoRoble.id_element == "node") {
        // En cas que sigui un node, recorrem els seus atributs
        for (int i = 0; i < viejoRoble.atributs.size(); i++) {
                // Iterem fins trobar el atribut que ens interessa
                if (viejoRoble.atributs[i].first == "timestamp") {
                        tempsValor = viejoRoble.atributs[i].second;
                }
        }
        // Recorrem els fills d'aquest node
        for (int i = 0; i < viejoRoble.fills.size(); i++) {
                // Iterem i avaluem tots els fills que son tags
                if (viejoRoble.fills[i].first == "tag") {
                        // Emmagatzemem el valor d'aquest tag
                        pair<string, string> valorTag =
                                Util::kvDeTag(viejoRoble.fills[i].second);
                        // Comprovem que es el tag que busquem
                        if (valorTag.first == "wheelchair") {
                                permetMobilitatReduida =
                                        (valorTag.second == "yes")? true : false;
                        }
                }
        }
}
```

TASCA 1: Imprimir per pantalla l'identificador, latitud (***lat***), longitud (***lon***), així com el nom (***name***), de l'element XML `viejoRoble` descrit en aquest apartat 2.1.

Cal tenir-ne en compte que, almenys, cal transformar latitud i longitud a tipus `double`.

Recordeu que en la solució de la pràctica haureu de recollir tots els punts d'interès, que per definició, són nodes que **no** són nodes de camí que tenen un atribut ***name***.

Implementeu aquesta Tasca 1 dins el projecte **P3-OSM-Tasca1-i-2-VS2022.sln**

Pel que fa a les vies, es representen de la forma següent:

```
<way id="31889094" visible="true" version="6" changeset="102251634"
 timestamp="2021-04-03T21:17:17Z" user="Interswd" uid="3414856">
  <nd ref="357125056"/>
  <nd ref="8591278795"/>
  <nd ref="357124693"/>
  <nd ref="357125054"/>
  <tag k="highway" v="living_street"/>
  <tag k="name" v="Carrer de Sant Joaquim"/>
  <tag k="oneway" v="yes"/>
  <tag k="surface" v="paving_stones"/>
 </way>
```

Els nodes integrants del `way` s'hauran llegit prèviament, i l'estructura `XmlElement` corresponent es construirà de la manera següent:

```
XmlElement camiJoaquim = {

        // id_element =
        "way",

        // atributs =
        {
                make_pair("id", "31889094"),
                make_pair("visible", "true"),
                make_pair("version", "6"),
                make_pair("changeset", "1002251634"),
                make_pair("timestamp", "2021 - 04 - 03T21:17 : 17Z"),
                make_pair("user", "Interswd"),
                make_pair("uid", "3414856")
        },

        // fills =
        {
                make_pair<string, vector<pair<string, string>>>
                        ("nd", {make_pair("ref", "357125056")}),
                make_pair< string, vector<pair<string, string>>>
                        ("nd", {make_pair("ref", "8591278795")}),
                make_pair< string, vector<pair<string, string>>>
                        ("nd", {make_pair("ref", "357124693")}),
                make_pair< string, vector<pair<string, string>>>
                        ("nd", {make_pair("ref", "357125054")}),
                make_pair<string, vector<pair<string, string>>>
                        ("tag", {
                                make_pair("k", "highway"),
```

```
                        make_pair("v", "living_street")
                    }),
                make_pair < string, vector<pair<string, string>>>
                    ("tag", {
                        make_pair("k", "name"),
                        make_pair("v", "Carrer de Sant Joaquim")
                    }),
                make_pair <string, vector<pair<string, string>>>
                    ("tag", {
                        make_pair("k", "oneway"),
                        make_pair("v", "yes")
                    }),
                make_pair <string, vector<pair<string, string>>>
                    ("tag", {
                        make_pair("k", "surface"),
                        make_pair("v", "paving_stones")
                    })
        }
};
```

TASCA 2: Imprimir per pantalla els identificadors dels nodes que formen el `XmlElement camiJoaquim`, així com el valor del atribut ***highway***. Implementeu aquesta tasca dins la solució.

Recordeu que en la solució de la pràctica només haureu de tenir en compte tots els camins que tenen el tag ***highway*** (independentment del valor que pugui tenir), així com saber quins nodes de tipus camí, formen cada camí.

Implementeu aquesta Tasca 2 al mateix projecte **P3-OSM-Tasca1-i-2-VS2022.sln**

2.2 CLASSE PUNTDEINTERES

Cal definir diferents classes per representar les diferents tipologies de punts d'interès mitjançant herència, afegint els atributs i mètodes necessaris a partir de la classe base **PuntDeInteresBase**. Es definirà un nom i un color per a la seva representació a la interfície gràfica. Tot i que es pot assignar qualsevol nom, es recomana utilitzar el valor proporcionat per l'etiqueta "name" dels nodes d'OSM.

D'altra banda, per a la selecció del color, caldrà utilitzar atributs addicionals de **PuntDeInteresBase** que caracteritzin el punt i assignar-hi un valor en format hexadecimal (per exemple, 0xFFFFFF o 0xFFA500).

A manera d'exemple, es pot definir una classe "Restaurant" que representi en color blau els establiments que ofereixen ***pizza***:

```
<node id="8606512476" visible="true" version="1" changeset="102562605"
 timestamp="2021-04-08T12:04:11Z" user="Interswd" uid="3414856"
 lat="41.4928550" lon="2.1426380">
  <tag k="amenity" v="restaurant"/>
  <tag k="cuisine" v="pizza"/>
  <tag k="name" v="El Colosseo"/>
 </node>
```

O bé definir una classe "PuntDeInteresBotiga" per a les entitats amb l'etiqueta ***shop*** que siguin accessibles per a persones en cadira de rodes:

```
<node id="2561518617" visible="true" version="2" changeset="48212270"
 timestamp="2017-04-28T00:25:16Z" user="Sioul" uid="832276" lat="41.4905682"
 lon="2.1447148">
  <tag k="addr:city" v="Cerdanyola del Vallès"/>
  <tag k="addr:housenumber" v="16"/>
  <tag k="addr:postcode" v="08290"/>
  <tag k="addr:street" v="Carrer González,"/>
  <tag k="name" v="Mobiprix Mobles Cerdanyola del Vallès"/>
  <tag k="operator" v="Mobiprix"/>
  <tag k="phone" v="+34 902 104 168"/>
  <tag k="shop" v="furniture"/>
  <tag k="website" v="http://www.mobiprix.com"/>
  <tag k="wheelchair" v="yes"/>
 </node>

<node id="4448289429" visible="true" version="5" changeset="102250867"
 timestamp="2021-04-03T20:47:04Z" user="Interswd" uid="3414856"
 lat="41.4929978" lon="2.1423720">
  <tag k="name" v="Casa Cerdanyola"/>
  <tag k="opening_hours" v="Mo-Fr 09:00-14:00,16:00-21:00, Sa 09:00-
 14:30,16:00-21:00"/>
  <tag k="shop" v="hardware"/>
  <tag k="wheelchair" v="yes"/>
 </node>
```

Tant el nom com el color seleccionats s'han de retornar mitjançant la implementació dels mètodes de la classe derivada:

```
virtual std::string getName();
virtual unsigned int getColor();
```

TASCA 3: Implementeu dues classes, una anomenada **PuntDeInteresBotigaSolucio** i una altra anomenada **PuntDeInteresRestaurantSolucio** a on implementar els mètodes **getName()** i **getColor()** de la classe **PuntDeInteresBase**.

El **PuntDeInteresBotigaSolucio** disposarà d'un atribut addicional que contindrà el valor del tag "shop", a on:

1. Retornarà el color 0xDFFF00 els que són "supermarket".
2. Retornarà el color 0xFF7F50 els que són "tobacco".
3. Retornarà el color 0xFA8072 els que són "bakery".
4. Retornarà el color 0x4CBB17 d'aquells "bakery" si tenen el "opening_hours" obert de "06:00-22:00" i tenen accés per persones amb mobilitat reduïda.

Nota: El valor pot ser "Div 06:00-22:00" o "Dill 06:00-22:00". Les dues entrades son vàlides. Utilitza el find de std::string.

5. Finalment, retornarà el color 0xFFEA00 en els altres casos (que tenen el tag "shopt" però que el seu valor no és dels esmentats).

El **PuntDeInteresRestaurantSolucio** disposarà de dos atributs addicionals, el tipus de cuina ("cuisine"), i si en facilita l'accés a les persones amb mobilitat reduïda, on:

1. Retornarà el color 0x7FFFD4 tots els restaurants que ofereixen "pizza" i faciliten l'accés a persones amb mobilitat reduïda.

2. Retornarà el color 0x00FFFF tots els restaurants que ofereixen "chinese" com a tipus de cuina.

3. Retorna el color 0x5D3FD3 tots els restaurants que faciliten l'accés a persones amb mobilitat reduïda.

4. Retornarà el color definit per defecte en els altres casos.

Nota: El color per defecte està definit a la classe **PuntDeInteresBase**, i pot ser canviat. En altres paraules, eviteu utilitzar el "valor de manera directe", i accediu al valor que té definit la classe base utilitzant els mecanismes d'herència.

2.3 CLASSE VIRTUAL PURA CAMIBASE

Així mateix, cal implementar una solució per a la gestió de les vies del mapa.

Amb aquest objectiu es proporciona la classe **CamiBase**, que és una classe abstracta (virtual pura). Caldrà definir una classe derivada que implementi el mètode *getCamiCoords()* a partir de la signatura següent:

```
class CamiBase {
	public:
		virtual std::vector<Coordinate> getCamiCoords() = 0;
};
```

Aquest mètode ha de retornar un vector de coordenades, on `Coordinate` és una estructura de dades amb dos camps, `lat` i `lon`, que defineixen la posició geogràfica d'un node de la via.

La classe **CamiSolucio** ha d'heretar de **CamiBase** i s'ha d'integrar dins de **MapaSolucio**.

D'aquesta manera, es podrà retornar fàcilment el vector d'instàncies de **CamiBase** (conjunt de totes les vies del mapa) mitjançant el mètode *getCamins()*.

TASCA 4: Com a prova per visualitzar un camí, definiu una classe anomenada **CamiSolucio** que implementi el mètode **getCamiCoords()** i que retorni el següent conjunt de coordenades, definit per (lat, lon):

```
(41.4928803, 2.1452381)
(41.4929072, 2.1452474)
(41.4933070, 2.1453852)
(41.4939882, 2.1456419)
```

NOTA: Això s'haurà de canviar al final perquè no siguin uns punts fixes sinó que es puguin obtenir a partir de les dades llegides.

2.4 CLASSE VIRTUAL PURA MAPABASE

Per a la visualització dels resultats i les consultes, es proporciona la interfície d'una classe que comunica amb la biblioteca dinàmica (DLL) que compilareu. Aquesta classe és una classe abstracta denominada **MapaBase**, la qual conté els membres virtuals purs que cal implementar.

Heu de crear la classe derivada de **MapaBase**, anomenada **MapaSolucio**, que emmagatzemarà la informació en estructures de dades internes. Aquestes estructures s'han de dissenyar i implementar de manera eficient per desar la informació extreta del fitxer XML (punts d'interès i vies formades per nodes).

A la classe derivada **MapaSolucio** cal implementar les dues funcions següents (cada funció es descriu a continuació, seguida de la tasca corresponent):

- **`void getPdis(std::vector<PuntDeInteresBase*>& pdis)`**
    - Que retornarà tot el conjunt de punts d'interès del mapa (supermercats, pastisseries, escoles, comissàries, etc.).

TASCA 5: Implementeu el mètode **getPdis(std::vector<PuntDeInteresBase*>&)** mitjançant la creació d'una classe derivada de **MapaBase** anomenada **MapaSolucio**, a on es compleixi que:

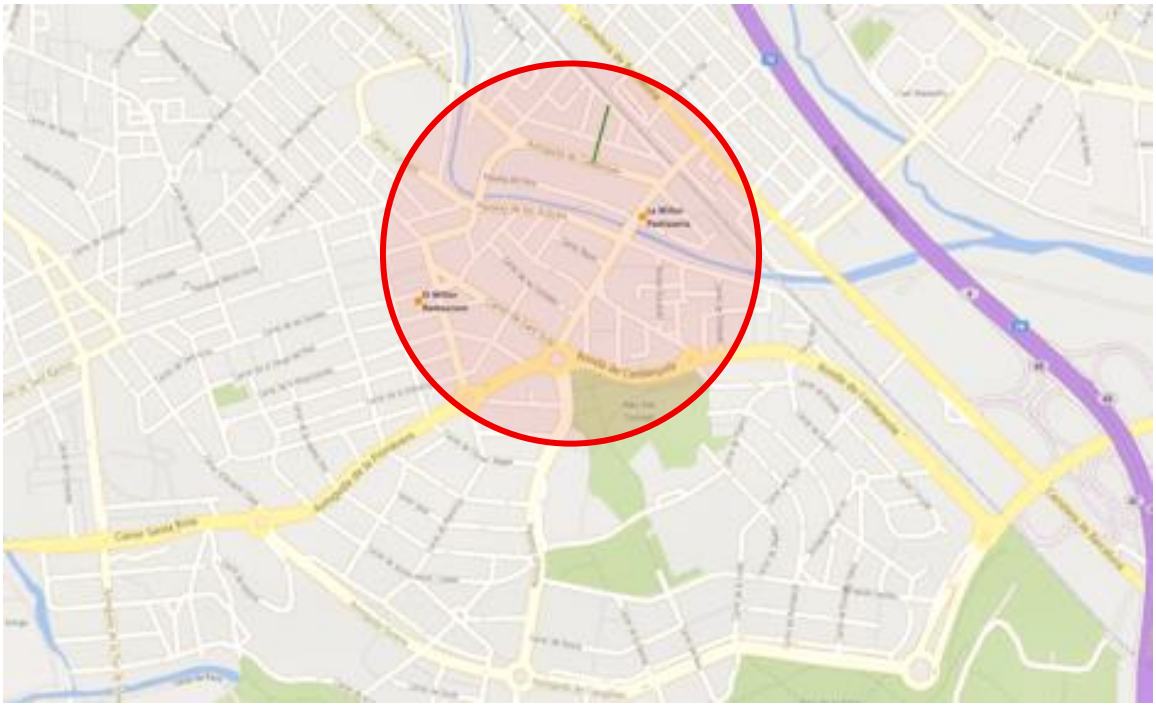
1. Retorna dos punts d'interès, un **PuntDeInteresBotigaSolucio** i un **PuntDeInteresRestaurantSolucio**, que són els que heu implementat a **Tasca 3**.
2. **PuntDeInteresBotigaSolucio** es un ***shop*** amb valor ***"bakery"*** anomenat "La Millor Pastisseria" situada a `(41.4918606, 2.1465411)`.
3. **PuntDeInteresRestaurantSolucio** es un ***amenity*** amb valor ***restaurant*** que ofereix cuina ***regional*** i que en permet l'accés a persones amb mobilitat reduïda. Aquest restaurant està situat a `(41.4902204, 2.1406477)`, i té el nom "El Millor Restaurant".
4. Finalment instancieu correctament la propietat **m_mapaBase** de la classe **MapaRender** per construir la vostra solució de **MapaSolucio**.

NOTA: Aquesta tasca més endavant haurà de retornar els punts d'interès del vector i no uns fixes.

- **`void getCamins(std::vector<CamiBase*>& camins)`**
    - Que retorna tots els camins que haureu parsejat i que són **highway**.
    - Nota: La **classe CamiBase** és una classe virtual pura i necessitareu definir una classe derivada **`CamiSolució`** a on afegireu nous atributs i funcionalitats.

TASCA 6: Retorneu el camí que ja heu implementat a la **Tasca 4** mitjançant la implementació de **getCamins(std::vector<CamiBase*>&)**

Un cop completades les tasques anteriors, es podran visualitzar els punts d'interès i les vies mitjançant la interfície gràfica, abans de processar cap fitxer OSM real. Si la implementació dels mètodes i atributs és correcta, el resultat mostrarà dos punts d'interès i una via a la regió de Cerdanyola del Vallès:



- **`void parsejaXmlElements(std::vector<XmlElement>& xmlElements)`**
    - Aquesta funció rep tots els nodes del fitxer XML llegits per la classe **MapaRender**.
    - A partir de l'anàlisi de cada element rebut com a paràmetre, s'ha de modificar l'estructura interna dissenyada per emmagatzemar la informació.

A partir d'aquest punt, es poden utilitzar les dades reals d'un fitxer OSM carregat:

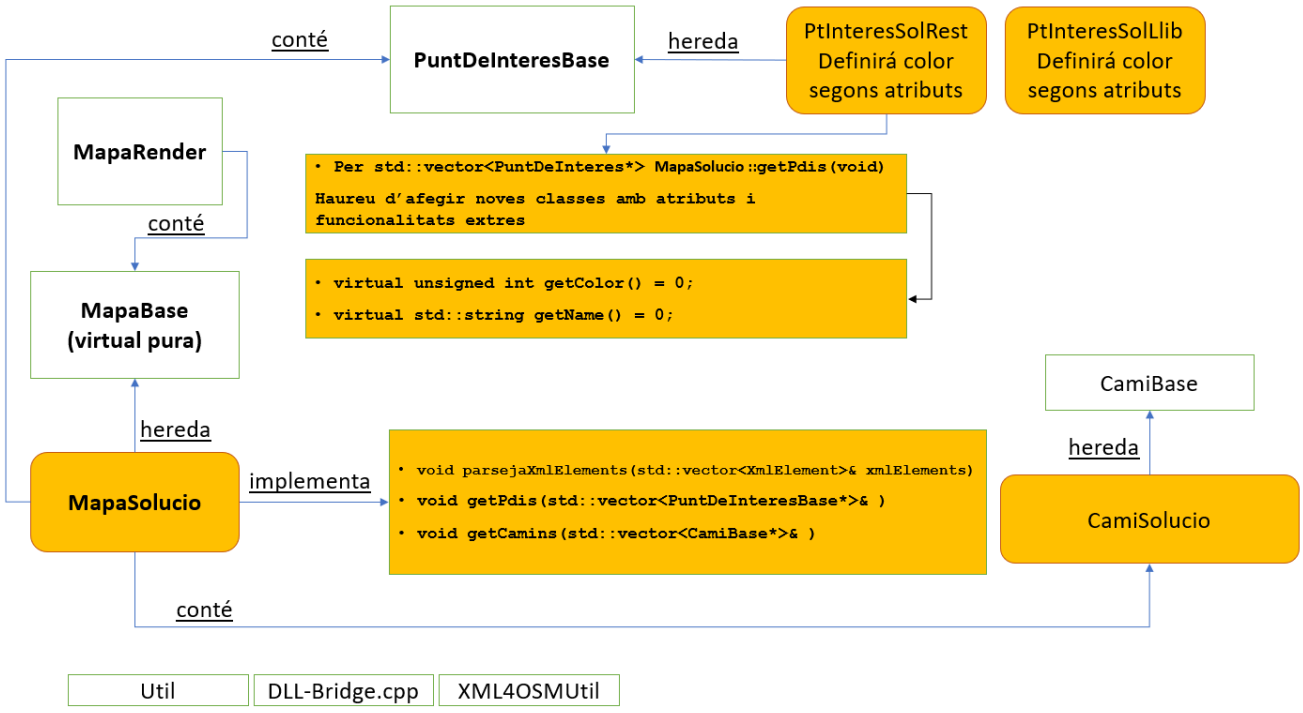
TASCA 7: Penseu en les estructures de dades, atributs i mètodes interns de **MapaSolucio** i **CamiSolucio** que cal tenir-ne per parsejar i emmagatzemar la informació de `xmlElements` a **parsejaXmlElements(…)** tenint en compte els exercicis realitzats a la **Tasca 1** i **Tasca 2**.

TASCA 8: Construir i retornar tots els Punts d'interès parsejats per `xmlElements` a través de la funció **getPdis(std::vector<PuntDeInteresBase*>&)**, utilitzant els colors descrits en la **Tasca 3**.

TASCA 9: Construir i retornar tots els camins parsejats per `xmlElements` a través de la funció **getCamins(std::vector<CamiBase*>&)**.

2.5 DIAGRAMA DE CLASSES

En aquesta secció es presenta un diagrama de classes que il·lustra les relacions entre els diferents components. En color taronja es mostren les classes que heu d'implementar per a la primera part de la pràctica, mentre que en blanc es representen els components ja proporcionats:



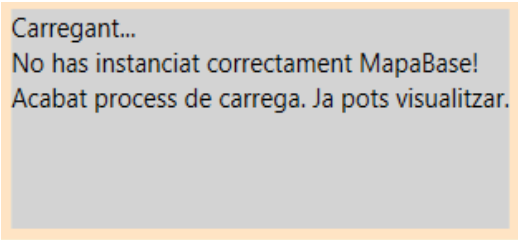
2.6 CLASSE UTIL I FITXER COMMON.H

Al fitxer de capçalera `Common.h` es troba la definició de les estructures **Coordinate** i **XmlElement**, així com els tipus **PAIR_ATTR_VALUE i CHILD_NODE**, necessaris per comprendre la representació de les coordenades del mapa i per iterar sobre els elements XML d'OSM, tal com s'ha descrit a la secció 2.1.

Els mètodes més destacats de la classe `Util` són els següents:

- **`Util::escriuEnMonitor(std::string text)`**
    - Aquesta funció permet escriure al fitxer de registre "Log.txt" el text passat com a paràmetre ***text***.

Cal tenir en compte que el text mostrat a l'àrea del monitor es combinarà amb la informació generada per la interfície gràfica. Per exemple:



En aquest exemple, el missatge indica que no s'ha instanciat correctament **MapaBase** (situació habitual amb el codi inicial abans de desenvolupar la solució). Quan es produeix aquesta situació, s'executa la crida següent:

```
Util::escriuEnMonitor("No has instanciat correctament MapaBase!");
```

- **`Util::kvDeTag(std::vector<PAIR_ATTR_VALUE>& atributsTag)`**
    - La funció **kvDeTag** permet extreure els subelements de tipus ***<tag>*** de manera senzilla.

Aquesta funció itera sobre els atributs del tag (és a dir, `k="clau" v="valor"`) i retorna un parell on el primer element correspon a la clau i el segon al valor. Com a paràmetre, cal passar-li el conjunt d'atributs del tag que es processa (vegeu l'exemple anterior a la Tasca 1).

2.7 CONSIDERACIONS FINALS: VISIÓ TÈCNICA INTERNA I CONSELLS VARIS

El projecte de la solució s'ha creat amb **Visual Studio 2022** (compatible amb les edicions Community i Professional) i s'estructura en dos subprojectes: un desenvolupat en C# .NET (P3-OSM-WPF-2022) i un altre en C++ (P3-OSM-DLL-2022), sobre el qual fareu les implementacions.

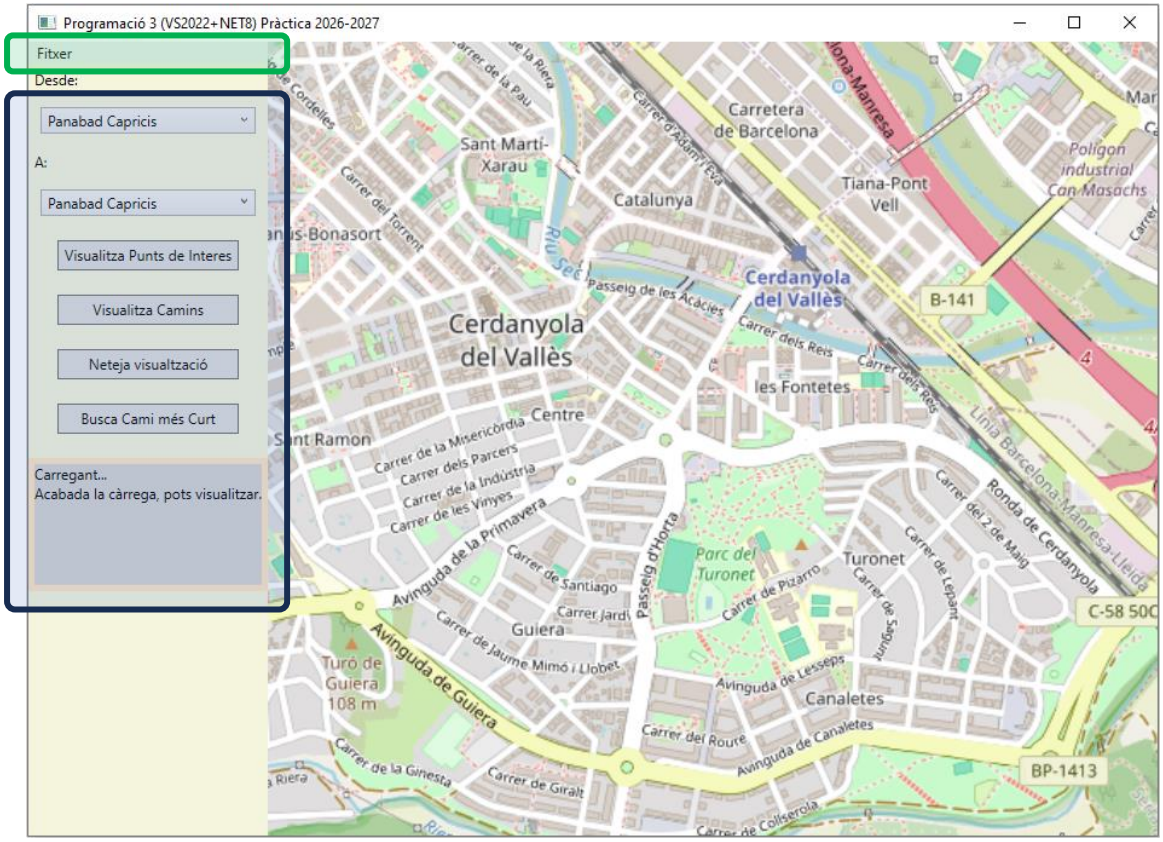
La comunicació entre ambdós projectes s'efectua mitjançant una biblioteca d'enllaç dinàmic (DLL). El projecte C++ genera una biblioteca que s'integra automàticament amb l'aplicació gràfica i n'ofereix les funcionalitats mitjançant les crides als mètodes definits a DLL-Bridge. D'aquesta manera, la part gràfica queda completament desacoblada del desenvolupament que heu de realitzar vosaltres.

Per treballar amb aquest entorn cal disposar de Visual Studio 2022 amb els components de C++ i C#, així com de l'entorn d'execució .NET 8 (que es descarregarà automàticament mitjançant Visual Studio si no està ja disponible dins el vostre IDE).

El projecte està configurat per a diferents arquitectures objectiu: X64 (per defecte), X86 i ARM64. Això permet el desenvolupament en entorns amb processadors diferents d'Intel/AMD, tot i que l'avaluació final s'executarà sempre sobre la plataforma x64. En equips amb CPU ARM cal seleccionar l'arquitectura ARM64; per a altres plataformes, seleccioneu x86 i activeu l'emulació de compatibilitat corresponent.

Atesa l'arquitectura del projecte, no és possible utilitzar std::cout de la STL per a la sortida de dades per pantalla. Per tant, cal emprar la funció Util::escriuEnMonitor() descrita prèviament, o bé utilitzar les eines de depuració (debugger) de l'entorn de desenvolupament.

Així mateix, no s'ha de modificar el fitxer DLL-Bridge.cpp ni cap altra classe que no s'hagi especificat explícitament en aquest document. A continuació es descriu l'arquitectura general del projecte.



La interfície gràfica d'usuari (GUI) es divideix en dos blocs principals:

1. Càrrega d'informació: accessible mitjançant Fitxer > Obrir.
2. Representació de la informació: mitjançant els botons de visualització.

En carregar un fitxer, el projecte P3-OSM-WPF-2022 inicia un fil d'execució secundari (BackgroundWorker_DoWork) que invoca el mètode loadOSMData(), definit a DLL-Bridge.cpp.

Aquest mètode crida la funció construeixOSM() de la classe **MapaRender**, la qual processa el XML mitjançant parseXML() de la classe **XML4OSMUtil** (que utilitza la biblioteca externa Xerces) i retorna el resultat com un vector d'elements XmlElement.

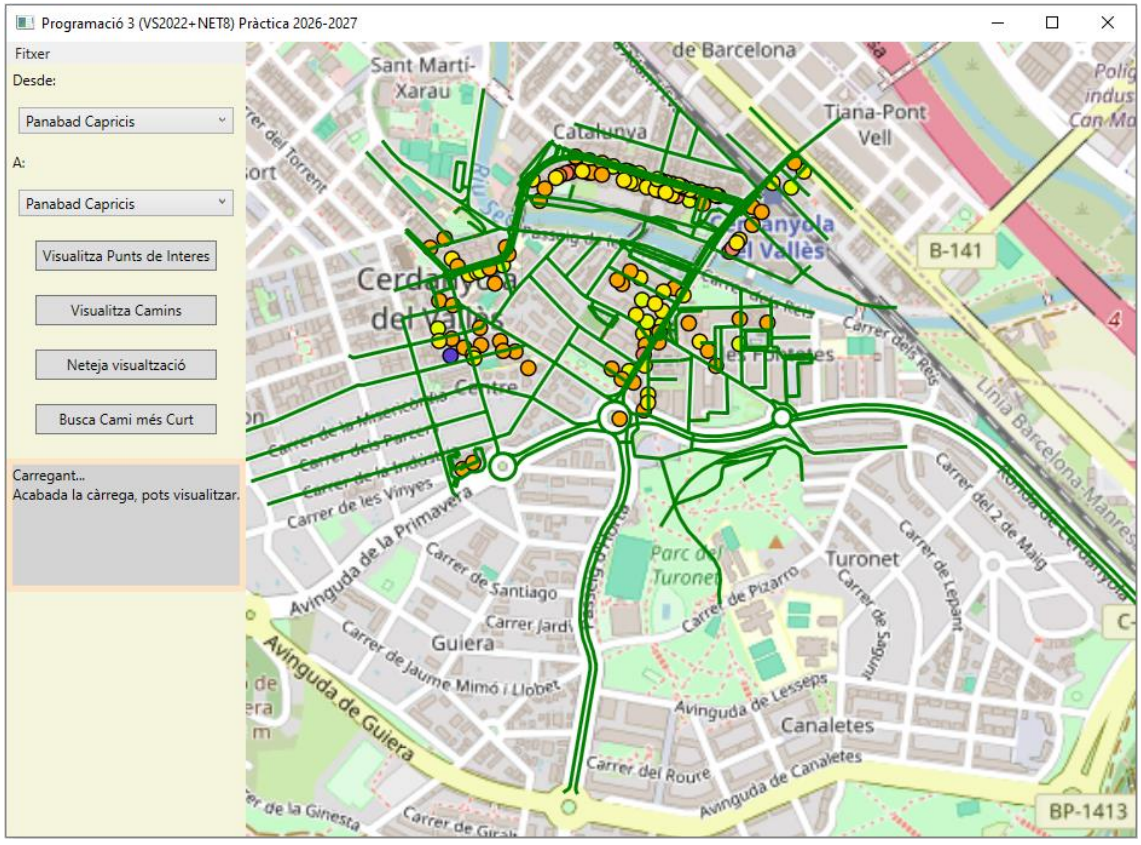
Aquest vector es passa com a paràmetre al mètode parsejaXmlElements() de la classe **MapaBase**, que heu d'implementar a la classe derivada **MapaSolucio**.

Quan finalitza la funció loadOSMData(), el mateix fil d'execució carrega les dades dels punts d'interès i de les vies mitjançant els mètodes `renderPois()` i `renderCamins()`.

Aquests mètodes invoquen les implementacions de getPdis(), getCamins() i getCamiCoords(), convertint les dades a les estructures **PoiBridge** i **WayBridge** per tal que puguin ser processades pel projecte en C#.

El fet que **MapaRender** apliqui el patró de disseny Singleton permet mantenir la persistència de les dades estructurades a parsejaXmlElements() entre les diferents crides a funcions. En cas contrari, la informació s'alliberaria en memòria en finalitzar l'àmbit de la funció **construeixOSM()**.

Un cop transferides les dades al projecte P3-OSM-WPF-2022, s'utilitza l'API de **MapControl** per representar la informació a la pantalla (colors, noms i coordenades dels punts d'interès, així com les coordenades de les vies), tal i com es pot veure en aquesta imatge:



La classe **XML4OSMUtil** i el conjunt de funcions contingudes a DLL-Bridge s'executen en segon pla per analitzar el fitxer XML i permetre representar les dades al mapa.

ATENCIÓ: Atesa l'estructura del projecte, la classe **XML4OSMUtil** i el mòdul **DLL-Bridge** són de caràcter intern. Per tant, les estructures i funcions bridge definides al fitxer de capçalera `Common.h` **no s'han de modificar en cap cas**.